

# National Textile University, Faisalabad



## Department of Computer Science

|                         |                                 |
|-------------------------|---------------------------------|
| <b>Name:</b>            | MUHAMMAD AHMAD SATTAR           |
| <b>Class:</b>           | BSCS 5 <sup>th</sup> B          |
| <b>Registration No:</b> | 23-NTU-CS-1059                  |
| <b>Assignment</b>       | Assignment: 2                   |
| <b>Course Name:</b>     | Embedded IoT Systems (CSE-3080) |
| <b>Submitted To:</b>    | Sir Nasir Mahmood               |
| <b>Submission Date:</b> | 17 - DEC - 2025                 |

# ASSIGNMENT 2

## QUESTION 1

### Part A: Short Questions

**1.What is the purpose of `WebServer server(80);` and what does port 80 represent?**

This line creates a web server on the ESP32.  
Port **80** is the default port used for HTTP websites, so browsers can open the ESP32 webpage without typing a port number.

**2.Explain the role of `server.on("/", handleRoot);` in this program.**

It tells the server what to do when a user opens the main page (/) of the website.  
When someone enters the ESP32 IP in a browser, the `handleRoot()` function is executed.

**3.Why is `server.handleClient();` placed inside the `loop()` function? What will happen if it is removed?**

It continuously checks for incoming client requests.  
If it is removed, the ESP32 will not respond to browser requests and the webpage will not load.

**4.In `handleRoot()`, explain the statement: `server.send(200, "text/html", html);`**

This sends a response to the browser.

- 200 means request successful
- "text/html" tells the browser the content type
- html is the webpage data being sent

**5.What is the difference between displaying last measured sensor values and taking a fresh DHT reading inside `handleRoot()`?**

Displaying last values is faster and avoids sensor delay.

Taking a fresh DHT reading inside `handleRoot()` can slow down page loading and may cause sensor errors if refreshed too often.

## **Part B: Long Question**

**Describe the complete working of the ESP32 webserver-based temperature and humidity monitoring system.**

### **ESP32 Wi-Fi Connection & IP Address Assignment**

The ESP32 connects to a Wi-Fi network using SSID and password. After successful connection, the router assigns an IP address, which is used to access the web page.

```
WiFi.begin(ssid, password);  
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
}  
Serial.println(WiFi.localIP());
```

## **Web Server Initialization & Request Handling**

A web server is created on port 80. The root URL (/) is linked with a function that sends the webpage to the browser.

```
WebServer server(80);  
server.on("/", handleRoot);  
server.begin();
```

Client requests are handled continuously inside the loop.

```
server.handleClient();
```

## **Button-Based Sensor Reading & OLED Update**

A button is used to take sensor readings only when needed. When pressed, the ESP32 reads temperature and humidity from the DHT sensor and updates the OLED display.

```
if (digitalRead(buttonPin) == LOW) {  
    temperature = dht.readTemperature();  
    humidity = dht.readHumidity();  
    displayValuesOnOLED();  
}
```

This prevents unnecessary sensor readings.

## Dynamic HTML Webpage Generation

The webpage is created using a string. Sensor values are added dynamically so updated data appears on the webpage.

```
String html = "<h1>ESP32 Sensor Data</h1>";  
html += "Temperature: " + String(temperature) + " C<br>";  
html += "Humidity: " + String(humidity) + " %";  
  
server.send(200, "text/html", html);
```

## Purpose of Meta Refresh

Meta refresh automatically reloads the webpage after a few seconds, showing updated sensor values without manual refresh. This improves real-time monitoring.

```
<meta http-equiv='refresh' content='5'>
```

## Common Issues & Their Solutions

- **Wi-Fi not connecting:** Check SSID/password and signal strength
- **Page not loading:** Ensure server.handleClient() is running
- **DHT sensor errors:** Avoid frequent readings and add delays
- **Slow response:** Do not read sensors inside every page request

## QUESTION 2

### Part A: Short Questions

**1. What is the role of Blynk Template ID in an ESP32 IoT project? Why must it match the cloud template?**

The Blynk Template ID links the ESP32 project with a specific dashboard layout on Blynk Cloud.

It must match the cloud template so the device knows which widgets and settings to use.

**2. Differentiate between Blynk Template ID and Blynk Auth Token.**

- **Template ID:** Identifies the project template on Blynk Cloud
- **Auth Token:** Authenticates a specific device and allows it to connect to the cloud

Template ID is for the project, Auth Token is for the device.

**3. Why does using DHT22 code with a DHT11 sensor produce incorrect readings? Mention one key difference between the two sensors.**

DHT11 and DHT22 use different data formats and accuracy ranges.

**Key difference:** DHT22 has higher accuracy and wider temperature range than DHT11.

#### **4. What are Virtual Pins in Blynk? Why are they preferred over physical GPIO pins for cloud communication?**

Virtual Pins are software-based pins used to send and receive data between ESP32 and Blynk widgets.

They are preferred because they are not tied to physical GPIO pins and work easily with cloud data.

#### **5. What is the purpose of using BlynkTimer instead of delay() in ESP32 IoT applications?**

BlynkTimer allows tasks to run at intervals without blocking the program. Using delay() stops Wi-Fi and cloud communication, while BlynkTimer keeps the system responsive.

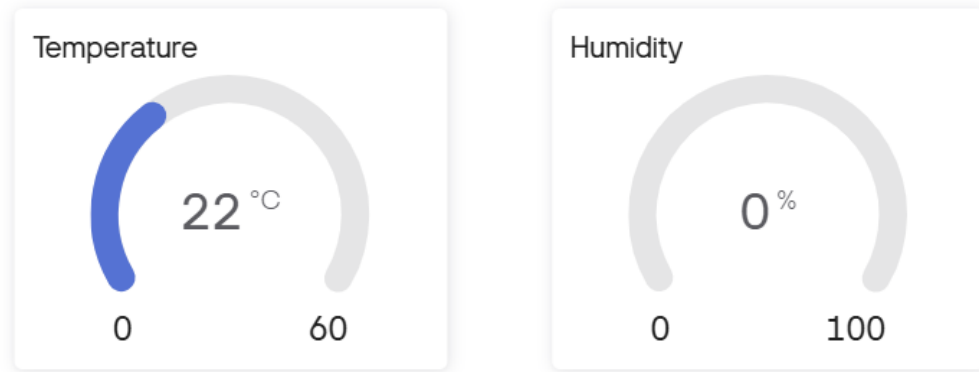
### **Part B: Long Question**

Explain the complete workflow of interfacing ESP32 with Blynk Cloud to display temperature and humidity values. Your answer should include

#### **Creation of Blynk Template and Datastreams**

First, a template is created on Blynk Cloud. Datastreams (Virtual Pins) are added for temperature and humidity so sensor data can be displayed on the mobile dashboard.

Example:



These datastreams define how data flows from ESP32 to the cloud.

### Template ID, Template Name, and Auth Token

- **Template ID** connects the ESP32 firmware to the correct Blynk Cloud template
- **Template Name** is used for project identification
- **Auth Token** authorizes the ESP32 device to communicate with Blynk Cloud

```
#define BLYNK_TEMPLATE_ID "TMPL6DMy_9p6o"  
#define BLYNK_TEMPLATE_NAME "DHT for Ahmad"  
#define BLYNK_AUTH_TOKEN "nxcbkVafNtE2mH2iD0whP969SkDz2Qwv"
```

All parameters must match the cloud configuration for successful connection.

### Sensor Configuration (DHT11 vs DHT22)

The sensor type must be correctly defined in the code. Using DHT22 configuration with a DHT11 sensor results in incorrect readings due to differences in accuracy and data format.

```
#define DHTTYPE DHT11
```



## Data Transmission Using Virtual Pins

Temperature and humidity values are read from the sensor and transmitted to Blynk Cloud using Virtual Pins.

```
Blynk.virtualWrite(V0, temperature);  
Blynk.virtualWrite(V1, humidity);
```

This enables real-time visualization on the Blynk application.

## Common Configuration Issues and Solutions

- **Connection failure:** Verify Wi-Fi credentials and Auth Token
- **No data on widgets:** Check Virtual Pin mapping
- **Incorrect readings:** Confirm correct DHT sensor type
- **Unstable connection:** Avoid delay(), use BlynkTimer for periodic tasks

```
BlynkTimer timer;  
timer.setInterval(2000L, sendData);
```

# DASHBORADS

## BLYNK CLOUD (WEB)

The screenshot shows the Blynk Console interface. On the left is a sidebar with navigation options: Get Started, Dashboards, Custom Data, Developer Zone (selected), Devices, Automations, Users, Organizations, Locations, Snapshots, Fleet Management, In-App Messaging, and Collapse. The main area is titled 'My organization - 7178PP'. Below this is a 'DEVELOPER ZONE' section with a list of items: My Templates (selected), Blueprints, Blynk Air (OTA), Static Tokens, Rule Engine, OAuth 2.0, Webhooks, and Integrations. The 'Templates' section is displayed, featuring a search bar and a single template card titled 'DHT for Ahmad' with '1 Device'. A '+ New Template' button is in the top right. At the bottom right, it says 'Region: SGP1' with links for 'Privacy Policy' and 'Terms of Service'.

This screenshot shows the 'DHT Sensor' device page in the Blynk Console. The sidebar is identical to the previous screenshot, with 'Developer Zone' selected. The main area shows the device details for 'DHT Sensor', which is 'Offline'. It includes a cube icon, a status bar with '20Wv', 'Ahmad', and 'My organization - 7178PP', and an 'Edit' button. Below this is a timeline selector with options: Live, 1h, 6h, 1d, 1w, 1mo, 3mo, 6mo, 1y, and All. Two large circular gauges are shown: 'Temperature' at 22 °C and 'Humidity' at 0 %. The bottom right corner displays 'Region: SGP1' with links for 'Privacy Policy' and 'Terms of Service'.

# BLYNK MOBILE APP

